

Passo a Passo — Remix IDE (Navegador)

Zero instalação. Tudo roda no navegador. Contratos compilados em segundos. Use este guia para a demonstração ao vivo na apresentação do projeto.

O que você vai demonstrar

Etapa	O que prova	Critério do desafio
Deploy dos contratos	Smart contracts funcionais na blockchain	Uso de blockchain
Cadastro de projeto	Ação de impacto registrada on-chain (GPS, espécie)	Registro verificável
Doação + TreeNFT	Certificado digital imutável para o doador	Rastreável
Declaração de plantio	Timestamp on-chain — prova que o plantio ocorreu	Auditável
Oracle aprova M0	Payout automático — contrato libera 10% sem burocracia	Mensurável
Oracle reprova projeto	Sistema registra honestamente a falha (estiagem)	Confiável
Refund pro-rata	Doador recupera automaticamente — sem depender de ninguém	Transparente

Pré-requisito — MetaMask (opcional, só para testnet)

Para rodar na blockchain local do Remix (recomendado para a demo) não precisa de MetaMask. Para publicar na Base Sepolia e mostrar no Basescan, instale a extensão MetaMask no navegador e adicione a rede:

Campo	Valor
Nome	Base Sepolia
RPC URL	https://sepolia.base.org

Chain ID	84532
Símbolo	ETH
Explorador	https://sepolia.basescan.org

Faucet para ETH de teste: faucet.quicknode.com/base/sepolia
Cole seu endereço MetaMask e receba ~0,01 ETH (suficiente para ~10 deploys).

Parte 1 — Carregar os contratos no Remix

1 Abrir o Remix

Acesse remix.ethereum.org no navegador. No painel esquerdo, clique em **File Explorer** (ícone de pasta).

2 Criar pasta de trabalho

Clique no ícone de pasta com "+" → nomeie `reforest`.
Dentro dela, crie outra pasta chamada `contracts`.

3 Carregar os 3 contratos (arquivos enviados junto com este guia)

Os arquivos `MockUSDC.sol`, `TreeNFT.sol` e `ReforestVault.sol` acompanham este material. Para subi-los ao Remix: no File Explorer, com a pasta `contracts` selecionada, clique no ícone

"Upload files"

(seta para cima, no topo do File Explorer) e selecione os três arquivos `.sol` de uma vez.

Alternativa: arraste os três arquivos do Explorer do Windows direto para dentro da pasta `contracts` no Remix.

Os imports `@openzeppelin/contracts` são resolvidos automaticamente pelo Remix via CDN — não precisa instalar nada.

Parte 2 — Compilar

4 Abrir o compilador

Clique no ícone **Solidity Compiler** (segundo ícone no menu lateral esquerdo).

5 Selecionar versão

Em "Compiler" selecione exatamente

0.8.24

(ou superior).

Marque

"Auto compile"

para compilar automaticamente ao salvar.

6 Definir o EVM como Cancun

Ainda no compilador, clique em

"Advanced Configurations"

.

No campo

EVM Version

selecione `cancun` (nas versões novas do Remix o valor `default` já é Cancun — pode deixar assim).

Por que isso importa.

O OpenZeppelin v5.1+ usa a instrução `mcopy` em `Bytes.sol`, introduzida na hard fork **Cancun**

e suportada só a partir do Solidity

0.8.24

. Se o compilador estiver em versão antiga ou com EVM `paris / shanghai`, a compilação falha com:

```
DeclarationError: Function "mcopy" not found.  
→ @openzeppelin/contracts/utils/Bytes.sol
```

A correção é exatamente esta: Solidity $\geq$ 0.8.24 + EVM `cancun`.

7 Compilar ReforestVault.sol

Com o arquivo `ReforestVault.sol` aberto, clique em

"Compile ReforestVault.sol"

.

Resultado esperado: ícone do compilador fica verde (sem erros).

Atenção ao fazer deploy: `mcopy` só funciona em redes que já passaram pela hard fork Cancun. Base Sepolia, Sepolia, Holesky e Ethereum mainnet já suportam. Por isso, no próximo passo, use o ambiente **"Remix VM (Cancun)"** — e não versões mais antigas da Remix VM (London, Paris, Shanghai).

Parte 3 — Deploy dos contratos

Clique no ícone **Deploy & Run Transactions** (quarto ícone, seta para baixo).

Environment: selecione "**Remix VM (Cancun)**" para demo local instantânea, ou "**Injected Provider - MetaMask**" para publicar na Base Sepolia.

8 Deploy MockUSDC

No dropdown "Contract" selecione `MockUSDC`.

Clique em

Deploy

(sem parâmetros).

Copie o endereço gerado em "Deployed Contracts" → chame de `USDC_ADDR`.

9 Deploy TreeNFT

Selecione `TreeNFT`.

No campo do constructor, cole o endereço da conta selecionada no topo (será o admin).

Clique em

Deploy

. Copie o endereço → `NFT_ADDR`.

10 Deploy ReforestVault

Selecione `ReforestVault`.

Preencha o constructor:

```
admin:           [endereço da sua conta]
paymentToken_:   [USDC_ADDR]
treeNft_:        [NFT_ADDR]
```

Clique em

Deploy

. Copie o endereço → `VAULT_ADDR`.

Parte 4 — Configurar papéis

11 Autorizar o oracle

Em "Deployed Contracts", expanda

ReforestVault

.

Função `setOracle` → cole o endereço da sua conta (será o oracle na demo).

Clique em

transact

.

12 Autorizar o Vault a mintar NFTs

Expanda

TreeNFT

.

Função `setMinter` → cole `VAULT_ADDR`.

Clique em

transact

.

13

Em ReforestVault, função `createProject` . Clique na setinha (v)

[illegible]**transact**

As aspas no `gpsCoords` são obrigatórias.

O valor `-19.9,-43.95` tem uma vírgula, e o Remix a interpreta como separador de argumentos — sem aspas a transação falha com `too many arguments (count=7, expectedCount=6)`. Digitando `"-19.9,-43.95"` entre aspas, o Remix trata tudo como um texto único.

budgetTotal: 10000000000 = 10.000 USDC (6 casas decimais: 10000×10^6). Lembre-se: createProject é onlyOwner — a conta selecionada no topo (dropdown

Account

) deve ser a

admin

que fez o deploy, não a conta colada no campo `planter`.

Parte 6 — Doação e emissão do TreeNFT

14 Mintar USDC para o doador

Mude a conta ativa no topo para uma segunda conta (será "Alice — doadora").

Em

MockUSDC

, função `mint`:

```
to:      [endereço da segunda conta]
amount: 30000000000
```

Clique em

transact

.

15 Aprovar o Vault para gastar o USDC

Ainda com a conta de Alice, em

MockUSDC

, função `approve`:

```
spender: [VAULT_ADDR]
amount:
1157920892373161954235709850086879078532699846656405640394575840079131296
39935
```

(aprovação ilimitada — simplifica a demo)

16 Fazer doação e mintar TreeNFT

Em

ReforestVault

, função `donate` :

```
projectId: 1
amount:    70000000000
mintNft:   true
```

Clique em

transact

. No log aparecem os eventos:

```
Donated: projectId=1 donor=0xAlice ... amount=70000000000 nftMinted=true
nftTokenId=1
TreeMinted: tokenId=1 projectId=1 donor=0xAlice ...
```

Alice recebeu um NFT com GPS e espécie gravados on-chain — certificado de impacto imutável.

Parte 7 — Declaração de plantio (timestamp on-chain)

17 Plantador declara o plantio

Mude para a conta do plantador (a segunda conta usada no `createProject`).

Em ReforestVault, função `declarePlanted` :

```
projectId: 1
```

Evento `Planted` é emitido com timestamp imutável.

A partir daqui os milestones começam a contar.

Parte 8 — Oracle satelital: milestone M0 aprovado

18 Oracle reporta 95% de sobrevivência

Volte para a conta do admin/oracle (conta 1).

Em ReforestVault, função `reportMilestone` :

```
projectId:      1
milestone:      0    (0=M0, 1=M6, 2=M12, 3=M36)
survivalBps:    9500
dataSourceHash:
0x2d3c1b4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b
```

Resultado esperado:

```
MilestoneReported: projectId=1 milestone=M0 survivalBps=9500
approved=true
PayoutReleased:   projectId=1 milestone=M0 amount=700000000
```

10% do orçamento (700 USDC) foi liberado automaticamente ao plantador, sem burocracia. O `dataSourceHash` é o SHA-256 da cena Sentinel-2 — qualquer auditor pode verificar.

Parte 9 — Simular um projeto que falhou (reprovação ao vivo)

Por que não dá para reprovar o M6 do projeto 1 aqui. O M6 só fica disponível 180 dias após o plantio (`M6_DELAY = 180 days` no contrato) e a **Remix VM não permite adiantar o relógio** — não existe botão "Change timestamp". Tentar reportar o M6 agora reverte com `"Milestone ainda nao disponivel"` .

A saída: demonstramos a reprovação ao vivo usando o **M0 de um segundo projeto** — o M0 está disponível na hora (`M0_DELAY = 0`). Reaproveitamos as **mesmas contas** da demo (admin/oracle e plantador): **não é preciso criar nenhuma conta nova**, só um novo projeto.

(O ciclo temporal completo M6 → M36 + refund é provado no Anvil, pela suíte de testes Foundry.)

19

No topo (dropdown

Account

), seleccione de novo a conta

admin

(a mesma que fez o deploy do ReforestVault).

Em ReforestVault, abra `createProject` pela

setinha (✓)

e preencha, simulando agora uma área de risco que não vingou:

[illegible]

Clique em

transact

. Por ser o segundo projeto, ele recebe automaticamente `projectId = 2` (o contador `nextProjectId` incrementa sozinho).

As duas ciladas de sempre: `createProject` é `onlyOwner` (a conta no topo precisa ser a **admin**) e o `gpsCoords` vai **entre aspas** por causa da vírgula.

20

Declarar o plantio do projeto 2 — conta do plantador

No topo, troque para a

conta do plantador

(a mesma segunda conta que você colocou no campo `planter`).

Em ReforestVault, função `declarePlanted` :

```
projectId: 2
```

Clique em

transact

. Isso grava o `plantedAt` do projeto 2 e, como o M0 tem delay zero, deixa o M0 disponível imediatamente.

Não precisa de doação no projeto 2 para esta demonstração: como o milestone vai ser *reprovado*, não há payout — o objetivo é só mostrar a trilha de falha.

21

Oracle reporta estiagem — 50% de sobrevivência — conta admin/oracle

No topo, volte para a

conta admin/oracle

(a conta 1).

Em ReforestVault, função `reportMilestone` :

```
projectId:      2
milestone:      0    (M0 — disponível na hora)
survivalBps:    5000
dataSourceHash:
0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b
```

Clique em

transact

. Resultado esperado:

```
MilestoneReported: projectId=2 milestone=M0 survivalBps=5000
approved=false
(nenhum PayoutReleased — 50% abaixo do threshold de 75%)
```

50% está abaixo do limite de 75% — milestone reprovado. Nenhum valor é liberado ao plantador. O sistema registra a falha com honestidade — sem esconder o problema. No projeto 1 o M0 foi aprovado e pagou; no projeto 2 o M0 foi reprovado e não pagou — as duas trilhas, ao vivo, no mesmo Remix.

Parte 10 — Verificar o NFT do doador

22 Consultar metadados do TreeNFT

Em

TreeNFT

, função `metadata` :

```
tokenId: 1
```

Resultado esperado:

```
projectId:    1
species:      Ipe-amarelo
gpsCoords:    -19.9,-43.95
plantedAt:    [timestamp do bloco]
originalDonor: 0xAlice ...
```

Esses dados são imutáveis na blockchain — nenhuma organização pode alterá-los.

Parte 11 — Publicar na testnet pública (Base Sepolia) pelo navegador

Por que fazer isso. A demo nas partes 1–10 roda na Remix VM (local), ótima para ensaiar. Mas o desafio valoriza um **contrato em testnet pública com endereço verificável** — é o que permite a banca auditar de fora. A boa notícia: é o *mesmo* fluxo, só muda o **Environment** e cada transação passa pela MetaMask gastando gás de teste.

23 Instalar a MetaMask e adicionar a Base Sepolia

Instale a extensão

MetaMask

no navegador (use uma carteira só de teste). Adicione a rede com os dados da seção "Pré-requisito — MetaMask" no início deste guia (RPC <https://sepolia.base.org>, Chain ID [84532](#)).

24 Pegar ETH de teste no faucet

Copie o endereço da sua conta MetaMask e peça ~0,02 ETH em faucet.quicknode.com/base/sepolia . É o que paga o gás — sem saldo o deploy falha com *"insufficient funds"*. 0,02 ETH cobre vários deploys + a demo inteira.

25 Trocar o Environment para a MetaMask

Na aba

Deploy & Run Transactions

, no campo

Environment

, troque [Remix VM \(Cancun\)](#) por

"Injected Provider — MetaMask"

. A MetaMask abre pedindo conexão — autorize e **confirme que ela está na rede Base Sepolia**

A partir daqui você está na blockchain pública:

toda

transação abre um popup da MetaMask para confirmar e gasta gás de teste. Confira a conta selecionada no topo do Remix — ela deve ser a sua conta MetaMask.

26 Deployar os 3 contratos (confirmando na MetaMask)

Mesma ordem das Partes 3–4, mas agora cada clique pede confirmação na MetaMask:

- 1) Deploy MockUSDC → confirmar na MetaMask
- 2) Deploy TreeNFT(admin) → confirmar
- 3) Deploy ReforestVault(admin, USDC_ADDR, NFT_ADDR) → confirmar
- 4) ReforestVault.setOracle(admin) e TreeNFT.setMinter(VAULT_ADDR) → confirmar

Anote os 3 endereços

que aparecem em "Deployed Contracts" — agora são endereços **públicos e auditáveis** na Base Sepolia.

27 Rodar o fluxo de impacto na testnet

Repita as Partes 5 a 9 (createProject → mint/approve → donate+NFT → declarePlanted → reportMilestone M0 aprovado → 2º projeto com M0 reprovado), confirmando cada transação na MetaMask.

Numa testnet real o relógio também não pula 180 dias — então M6/M12/M36 continuam indisponíveis. Use o mesmo truque já validado: a reprovação é demonstrada com o **M0 de um segundo projeto** (delay zero).

28 (Opcional) Verificar o código no explorer

Para o contrato aparecer "verificado" no Basescan (a banca lê o código direto lá): use o plugin

"Contract Verification"

do Remix com uma API key gratuita de basescan.org/myapikey, ou publique no **Sourcify**

pelo próprio Remix após compilar. Funciona sem isso, mas o código verificado fortalece a auditabilidade.

Parte 12 — Verificar no Basescan (se usou MetaMask)

Se fez o deploy na Base Sepolia, acesse [https://sepolia.basescan.org/address/\[VAULT_ADDR\]#events](https://sepolia.basescan.org/address/[VAULT_ADDR]#events) e mostre os eventos emitidos em tempo real.

O que mostrar	Onde encontrar no Basescan
Código verificado do contrato	Aba "Contract" → "Code"
Eventos on-chain	Aba "Contract" → "Events"

TreeNFT mintado para Alice	TreeNFT → aba "Token Transfers"
dataSourceHash do oracle	Evento MilestoneReported → campo dataSourceHash

Prova de auditabilidade (Windows PowerShell):

```
$texto = "S2A_MSIL2A_20260401T130251_N0500_R110_T23KPQ_20260401T163512"
$stream = [IO.MemoryStream]::new([Text.Encoding]::ASCII.GetBytes($texto))
Get-FileHash -InputStream $stream -Algorithm SHA256 | Select-Object
-ExpandProperty Hash
```

O hash gerado deve ser idêntico ao `dataSourceHash` gravado on-chain (em letras maiúsculas no PowerShell — compare ignorando o caso).

Resumo dos critérios demonstrados

Critério	Como foi demonstrado
Auditável	SHA-256 da cena Sentinel-2 gravado no evento — reproduzível por qualquer um
Verificável	Metadados do NFT consultáveis on-chain; hash conferível via sha256sum
Transparente	Todos os eventos públicos; código verificado no Basescan
Confiável	Apenas oracle autorizado reporta; setOracle impede outros
Mensurável	survivalBps em basis points; threshold 75% (7500); payout proporcional
Rastreável	TreeNFT com GPS, espécie e doador original gravados no bloco